

# Cryptography and Network Security

Course Code: **CSEG3040**

Instructor: [AMIT GURUNG](#)

Email: [amit.gurung@ddn.upes.ac.in](mailto:amit.gurung@ddn.upes.ac.in)

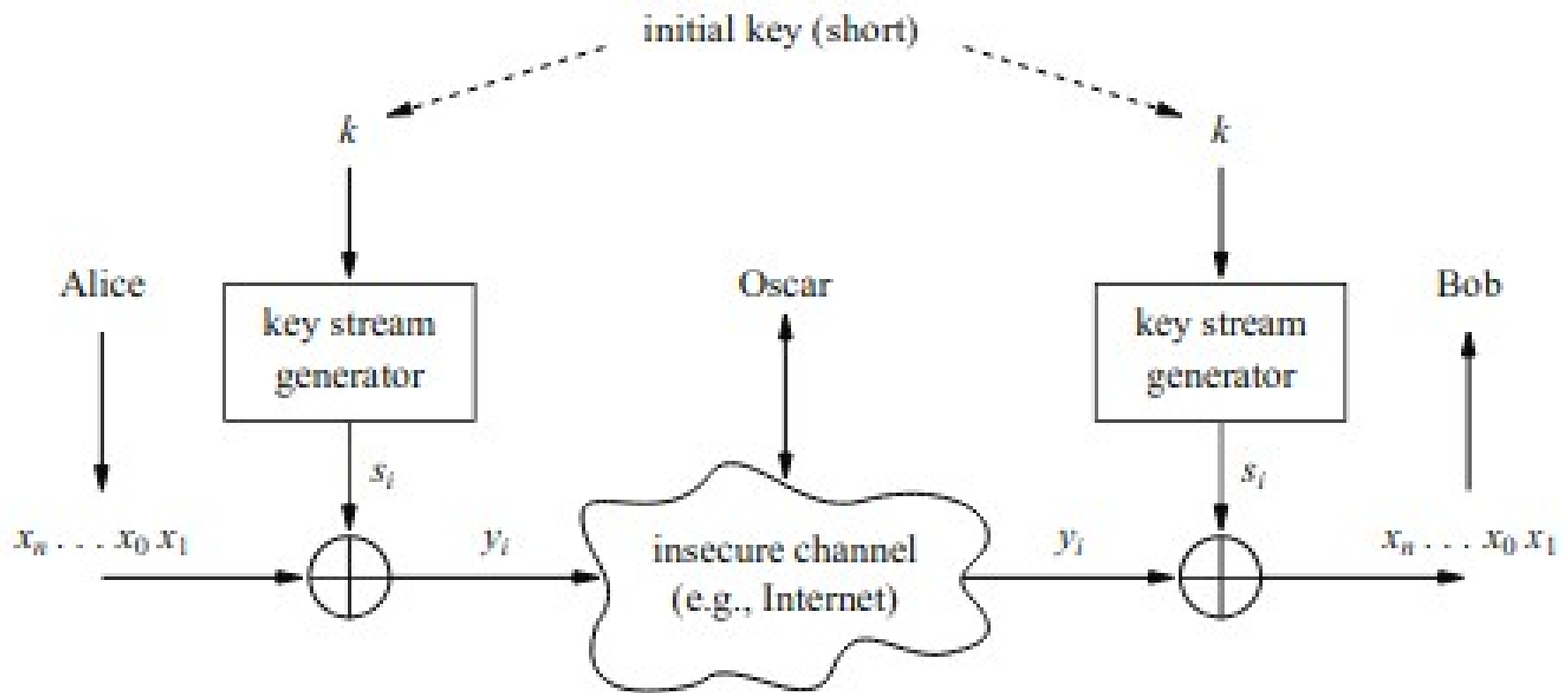


Aug – Dec, 2025

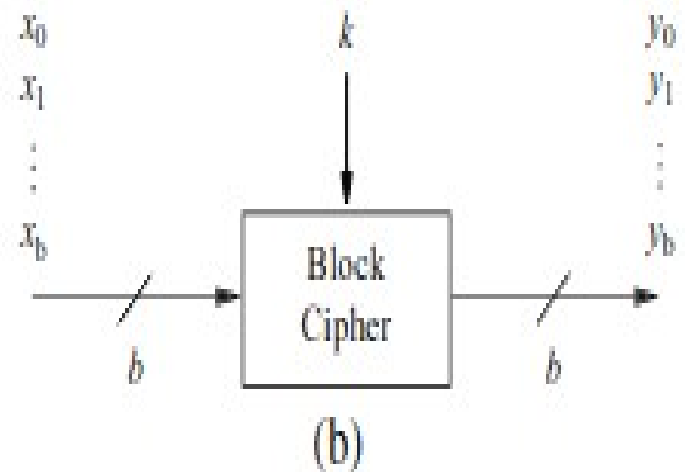
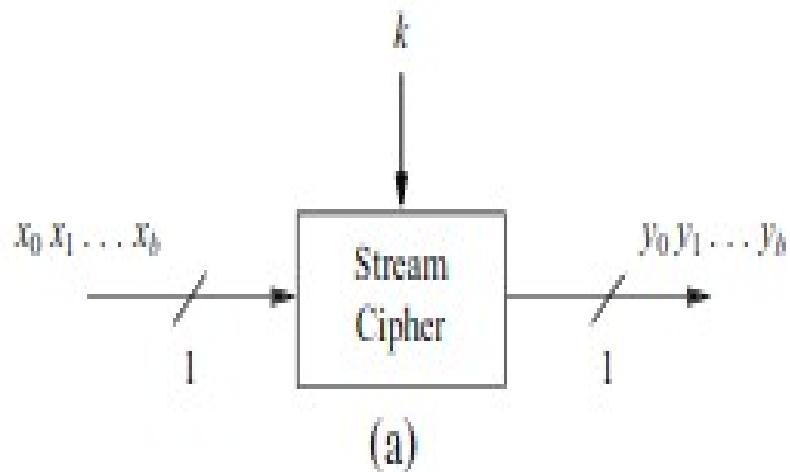
# Block vs Stream Ciphers

- block ciphers process messages in into blocks, each of which is then en/decrypted
- like a substitution on very big characters
  - 64-bits or more
- stream ciphers process messages a bit or byte at a time when en/decrypting
- many current ciphers are block ciphers
- hence are focus of course

# Stream Ciphers



# Block vs Stream Ciphers



# Stream Ciphers

## 2.1.2 Encryption and Decryption with Stream Ciphers

As mentioned above, stream ciphers encrypt plaintext bits individually. The question now is: How does encryption of an individual bit work? The answer is surprisingly simple: Each bit  $x_i$  is encrypted by adding a secret key stream bit  $s_i$  modulo 2.

### **Definition 2.1.1** Stream Cipher Encryption and Decryption

*The plaintext, the ciphertext and the key stream consist of individual bits,*

*i.e.,  $x_i, y_i, s_i \in \{0, 1\}$ .*

**Encryption:**  $y_i = e_{s_i}(x_i) \equiv x_i + s_i \pmod{2}$ .

**Decryption:**  $x_i = d_{s_i}(y_i) \equiv y_i + s_i \pmod{2}$ .

| $x_i$ | $s_i$ | $x_i + s_i \pmod{2}$ |
|-------|-------|----------------------|
| 0     | 0     | 0                    |
| 0     | 1     | 1                    |
| 1     | 0     | 1                    |
| 1     | 1     | 0                    |

Can we achieve this ciphertext ( $x_i + s_i \pmod{2}$ ) using some logic gate operation?

# Stream Ciphers

## Encryption by Sender

$x_0, \dots, x_6 = 1000001 = A$

$s_0, \dots, s_6 = 0101100$  XOR

---

$y_0, \dots, y_6 = 1101101 = m$

**Remember:** The strength of a stream cipher lies in **truly random key generation** — it must be **unpredictable** to ensure security.

## Decryption by Receiver

$y_0, \dots, y_6 = 1101101 = m$

$s_0, \dots, s_6 = 0101100$  XOR

---

$x_0, \dots, x_6 = 1000001 = A$

### Note:

Modulo 2 is same as XOR.

Check truth table

# Stream Ciphers

## Why Are Encryption and Decryption the Same Function?

The reason for the similarity of the encryption and decryption function can easily be shown. We must prove that the decryption function actually produces the plaintext bit  $x_i$  again. We know that ciphertext bit  $y_i$  was computed using the encryption function  $y_i \equiv x_i + s_i \pmod{2}$ . We insert this encryption expression in the decryption function:

$$\begin{aligned} d_{s_i}(y_i) &\equiv y_i + s_i \pmod{2} \\ &\equiv (x_i + s_i) + s_i \pmod{2} \\ &\equiv x_i + s_i + s_i \pmod{2} \\ &\equiv x_i + 2s_i \pmod{2} \\ &\equiv x_i + 0 \pmod{2} \\ &\equiv x_i \pmod{2} \quad Q.E.D. \end{aligned}$$

The trick here is that the expression  $(2s_i \pmod{2})$  has always the value zero since  $2 \equiv 0 \pmod{2}$ . Another way of understanding this is as follows: If  $s_i$  has either the value 0, in which case  $2s_i = 2 \cdot 0 \equiv 0 \pmod{2}$ . If  $s_i = 1$ , we have  $2s_i = 2 \cdot 1 = 2 \equiv 0 \pmod{2}$ .

# Block Cipher Principles

- Most symmetric block ciphers are based on a **Feistel Cipher Structure**
- Needed since must be able to **decrypt** ciphertext to recover messages efficiently
- Block ciphers look like an extremely large substitution
- Would need table of  $2^{64}$  entries for a 64-bit block
- Instead create from smaller building blocks
- Using idea of a product cipher

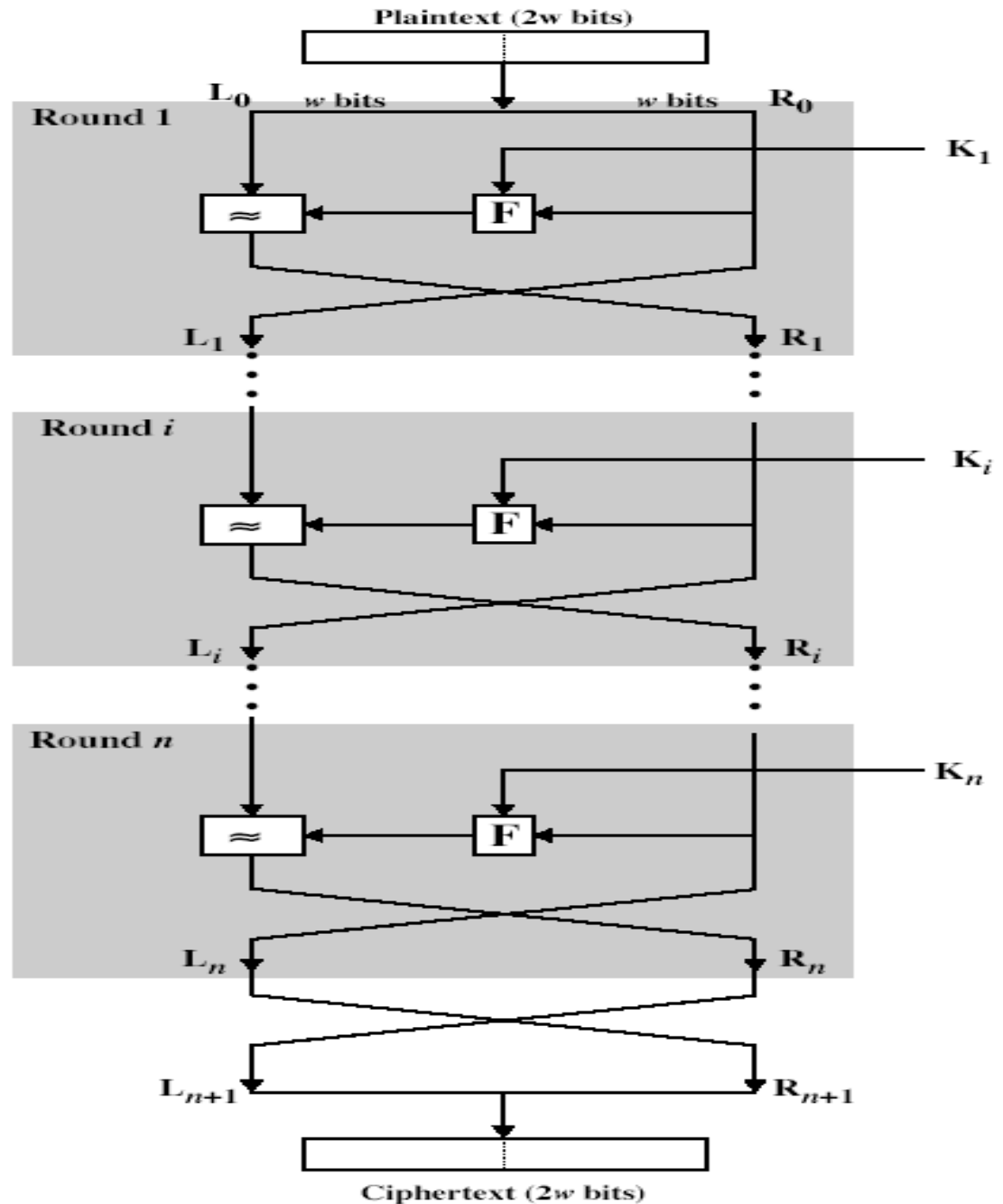
# Confusion and Diffusion

- cipher needs to completely obscure statistical properties of original message
- A one-time pad does this
- more practically **Shannon** suggested combining elements to obtain:
- **diffusion** – dissipates statistical structure of plaintext over bulk of ciphertext
- **confusion** – makes relationship between ciphertext and key as complex as possible

# Feistel Cipher Structure

- **Horst Feistel** devised the **feistel cipher**
  - based on **concept of invertible product cipher**
- Partitions **input block into two halves**
  - process through **multiple rounds** which
  - **perform a substitution on left data half**
  - based on **round function of right half & subkey**
  - then have **permutation swapping halves**
- implements **Shannon's substitution-permutation network concept**

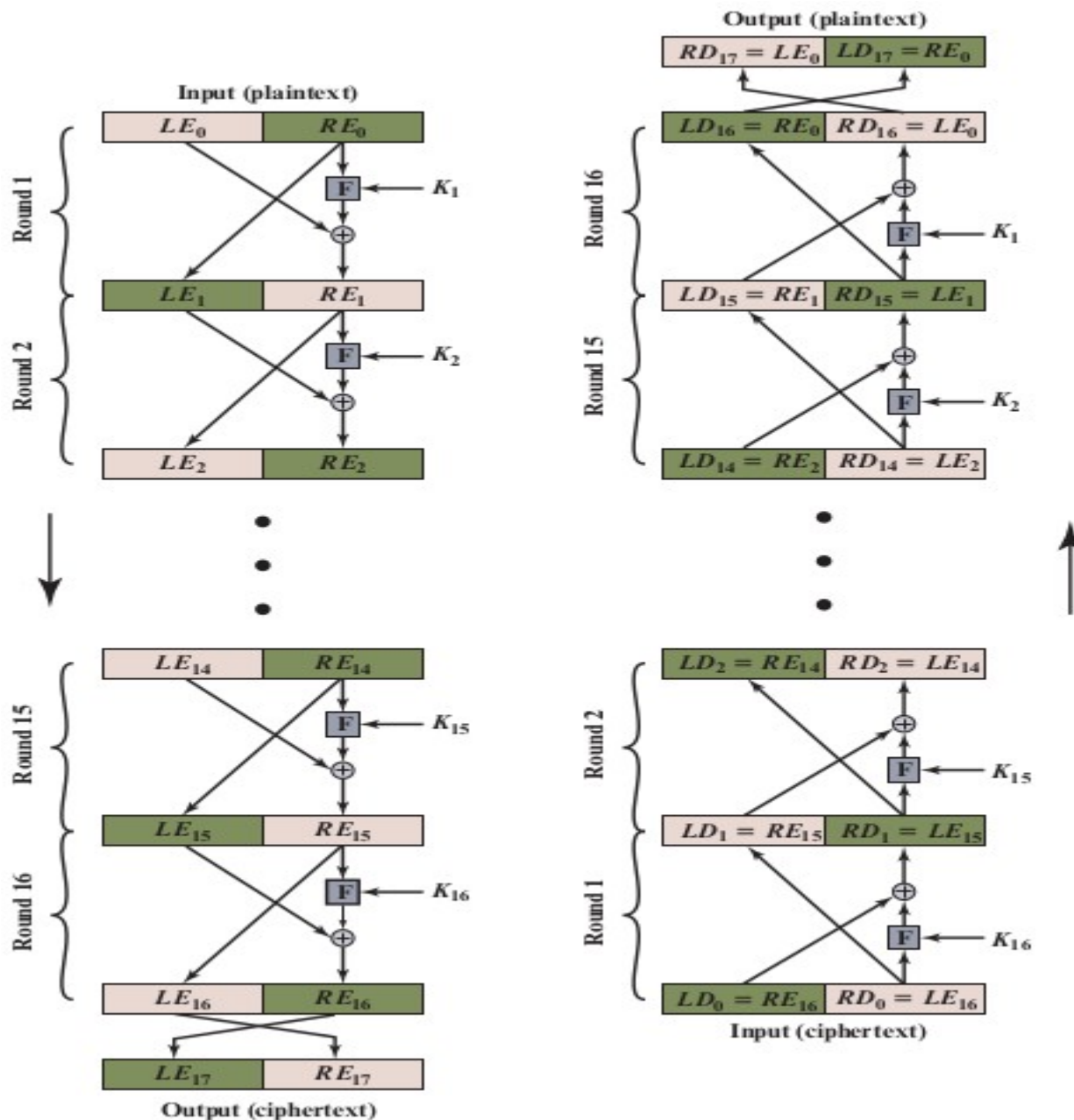
# Feistel Cipher Structure



# Feistel Cipher Design Principles

- **block size**
  - increasing size improves security, but slows cipher
- **key size**
  - increasing size improves security, makes exhaustive key searching harder, but may slow cipher
- **number of rounds**
  - increasing number improves security, but slows cipher
- **subkey generation**
  - greater complexity can make analysis harder, but slows cipher
- **round function**
  - greater complexity can make analysis harder, but slows cipher
- **fast software en/decryption & ease of analysis**
  - are more recent concerns for practical use and testing

# Feistel Cipher Decryption



# Data Encryption Standard (DES)

- Most widely used block cipher in world  
adopted in **1977 by National Bureau of Standards (NBS now NIST)**
- published on 15 January 1977 as **FIPS PUB 46**
- **Encrypts 64-bit data** using **56-bit key**
- Has widespread use
- Has been **considerable controversy** over its security

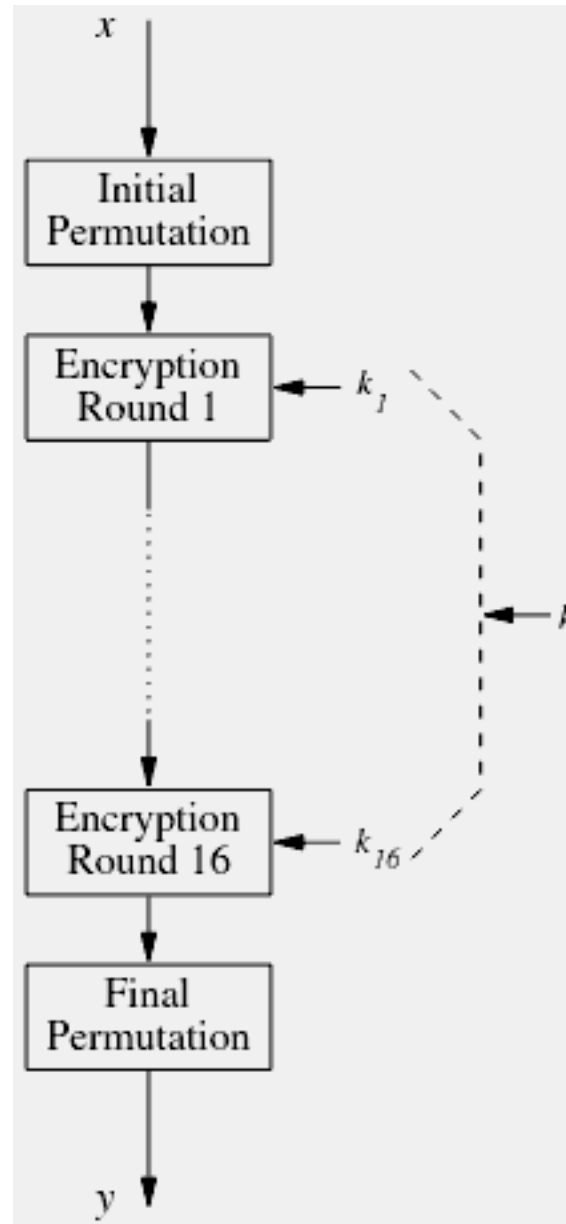
# DES History

- IBM developed **Lucifer cipher**
  - by team led by **Horst Feistel**
  - used **64-bit data** blocks with **128-bit key**
- then redeveloped as a commercial cipher with input from NSA and others
- in 1973 **NBS issued request for proposals for a national cipher standard**
- **IBM submitted their revised Lucifer which was eventually accepted as the DES**

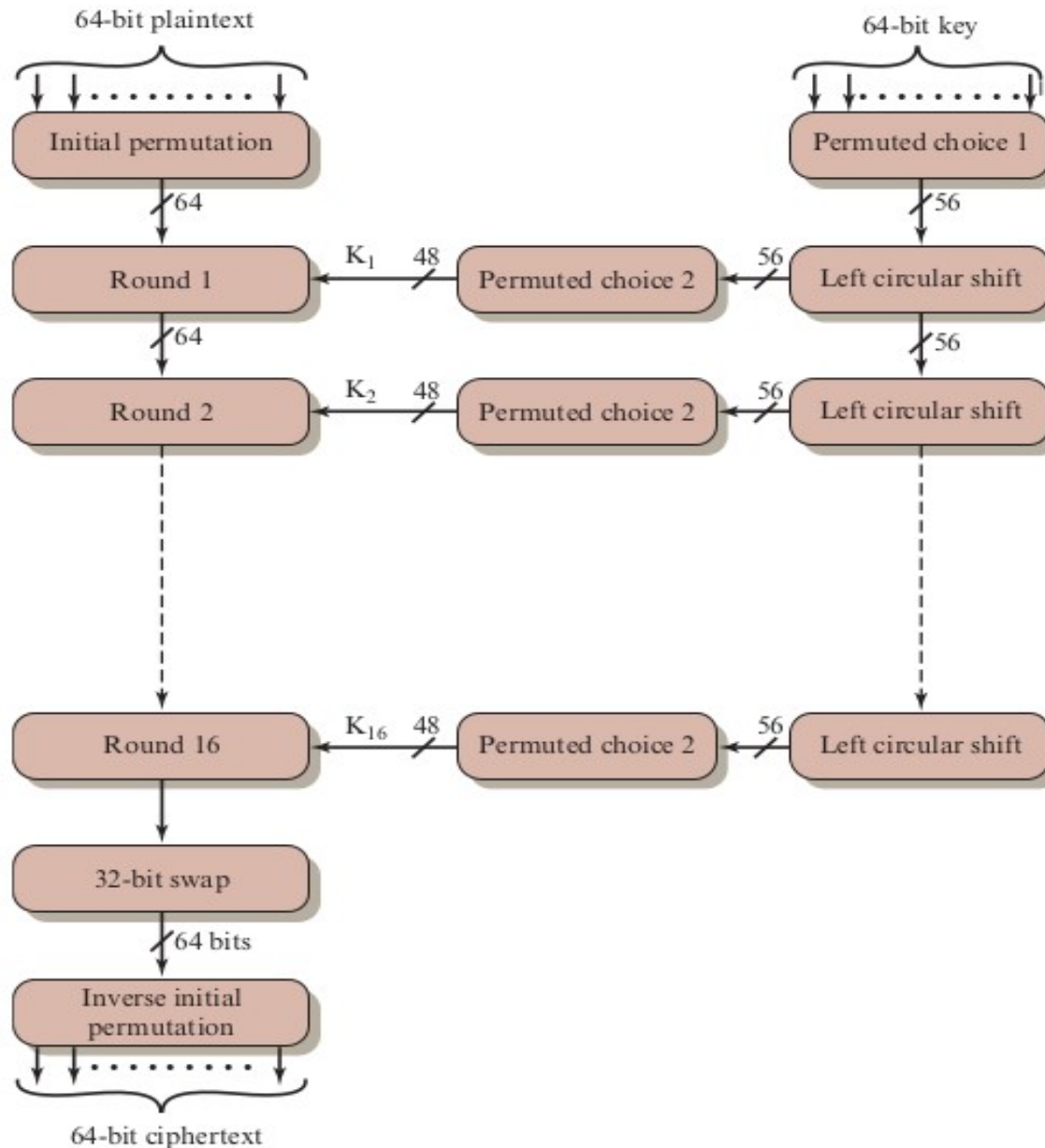
# DES Design Controversy

- although DES standard is public
- was considerable controversy over design
  - in **choice of 56-bit key** (vs Lucifer **128-bit**)
  - and because design criteria were classified
- subsequent events and public analysis show in fact design was appropriate
- DES has become widely used, esp in financial applications

# Overview of the **DES** Algorithm



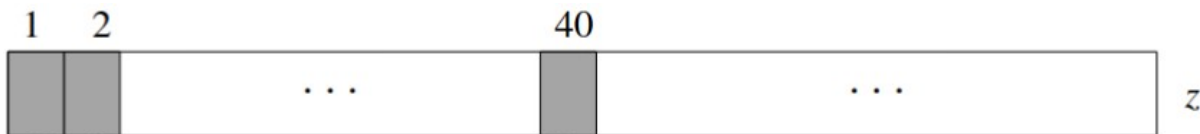
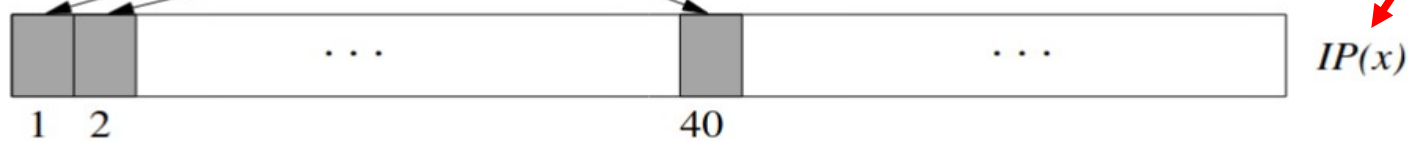
# DES Encryption



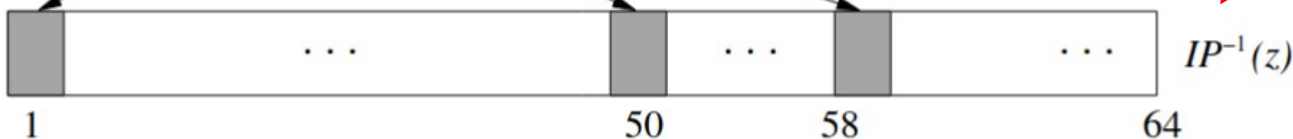
# Initial and Final Permutation



bit swaps of the **initial permutation**



bit swaps of the **final permutation**

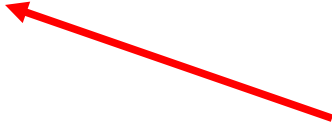


# Initial Permutation IP

- first step of the data computation
- ***IP reorders the input data bits***
- *even bits to LH half, odd bits to RH half*
- quite regular in structure (easy in h/w)

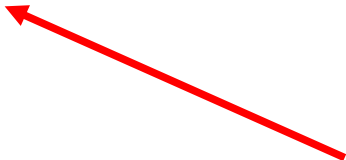
# Initial and Final Permutation

| $IP$ |    |    |    |    |    |    |   |
|------|----|----|----|----|----|----|---|
| 58   | 50 | 42 | 34 | 26 | 18 | 10 | 2 |
| 60   | 52 | 44 | 36 | 28 | 20 | 12 | 4 |
| 62   | 54 | 46 | 38 | 30 | 22 | 14 | 6 |
| 64   | 56 | 48 | 40 | 32 | 24 | 16 | 8 |
| 57   | 49 | 41 | 33 | 25 | 17 | 9  | 1 |
| 59   | 51 | 43 | 35 | 27 | 19 | 11 | 3 |
| 61   | 53 | 45 | 37 | 29 | 21 | 13 | 5 |
| 63   | 55 | 47 | 39 | 31 | 23 | 15 | 7 |



bit swaps of the **initial permutation**

| $IP^{-1}$ |   |    |    |    |    |    |    |
|-----------|---|----|----|----|----|----|----|
| 40        | 8 | 48 | 16 | 56 | 24 | 64 | 32 |
| 39        | 7 | 47 | 15 | 55 | 23 | 63 | 31 |
| 38        | 6 | 46 | 14 | 54 | 22 | 62 | 30 |
| 37        | 5 | 45 | 13 | 53 | 21 | 61 | 29 |
| 36        | 4 | 44 | 12 | 52 | 20 | 60 | 28 |
| 35        | 3 | 43 | 11 | 51 | 19 | 59 | 27 |
| 34        | 2 | 42 | 10 | 50 | 18 | 58 | 26 |
| 33        | 1 | 41 | 9  | 49 | 17 | 57 | 25 |



bit swaps of the **final permutation**

# DES Round Structure

- Uses two 32-bit L & R halves
- As for any Feistel cipher can describe as:

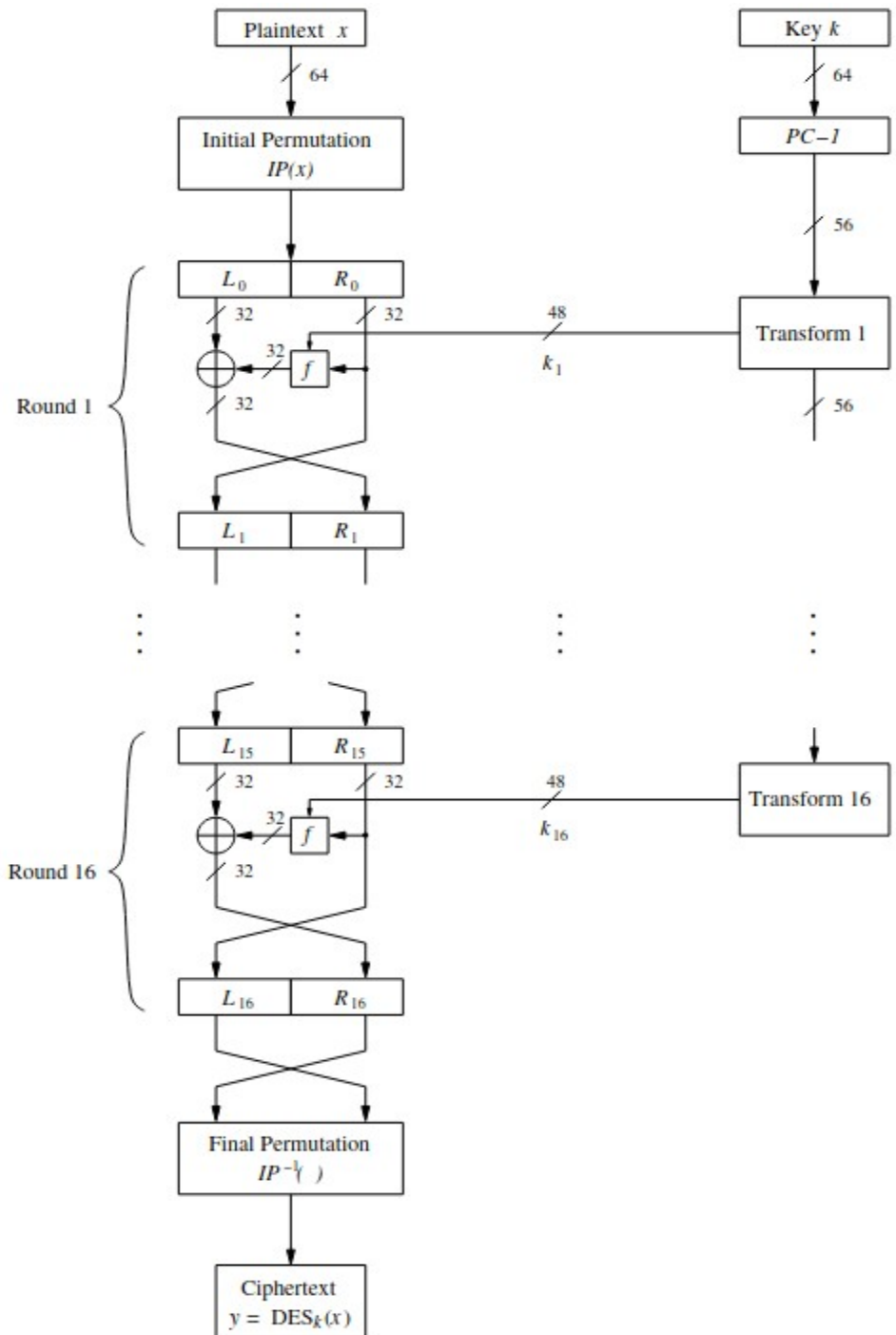
$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \text{ xor } F(R_{i-1}, K_i)$$

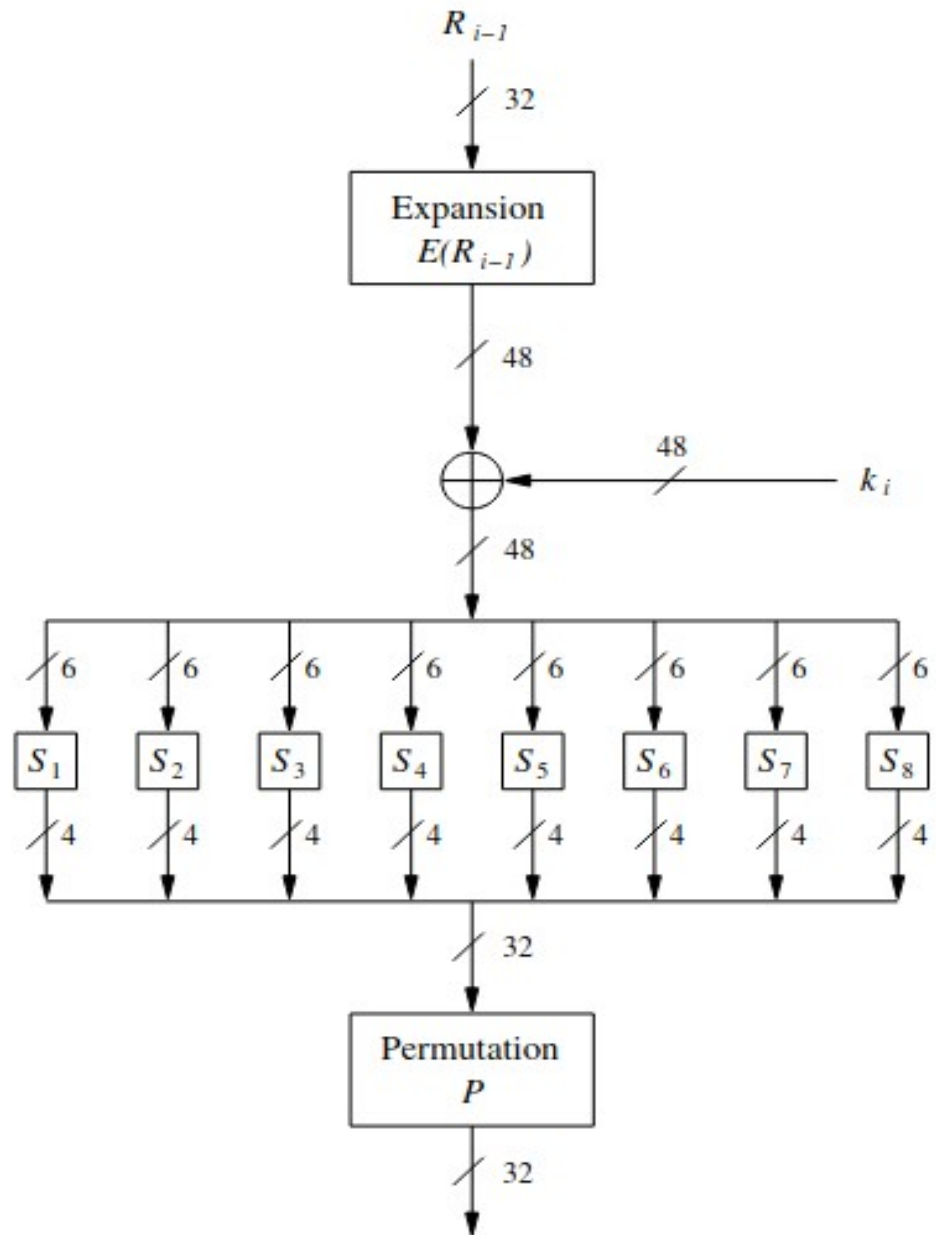
- Takes 32-bit R half and 48-bit subkey and:
  - First expands R to 48-bits using perm E
  - XOR's with the subkey
  - Then, passes through 8 S-boxes to get 32-bit result
  - Finally permutes this using 32-bit perm P

# The Feistel structure of DES

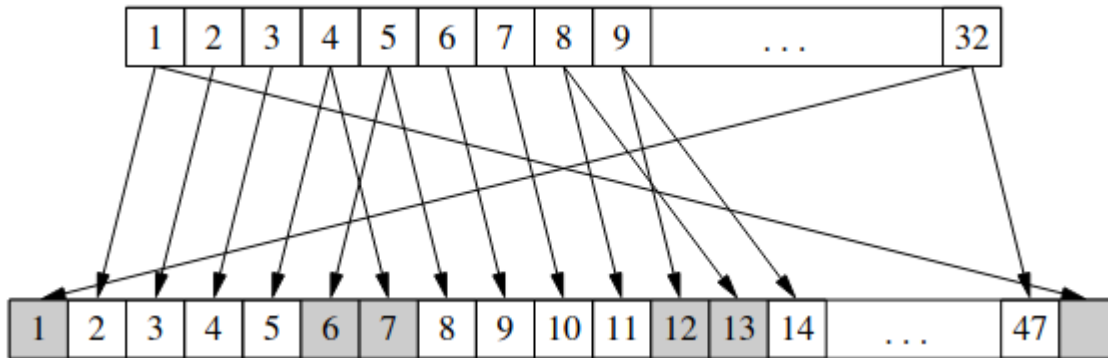
Each round is identical



# Block diagram of the $f$ -function



# E-box, a special type of permutation



| $E$ |    |    |    |    |    |
|-----|----|----|----|----|----|
| 32  | 1  | 2  | 3  | 4  | 5  |
| 4   | 5  | 6  | 7  | 8  | 9  |
| 8   | 9  | 10 | 11 | 12 | 13 |
| 12  | 13 | 14 | 15 | 16 | 17 |
| 16  | 17 | 18 | 19 | 20 | 21 |
| 20  | 21 | 22 | 23 | 24 | 25 |
| 24  | 25 | 26 | 27 | 28 | 29 |
| 28  | 29 | 30 | 31 | 32 | 1  |

bit swaps of the expansion function  $E$

- Exactly 16 of the 32 input bits appear twice in the output.

- Expansion causes diffusion.

# S-boxes

Each S-box is a lookup table that **maps a 6-bit input to a 4-bit output**.

Larger tables would have been cryptographically better, but they also become much larger;

Eight 4-by-6 tables were probably close the maximum size which could be fit on a single integrated circuit in 1974.

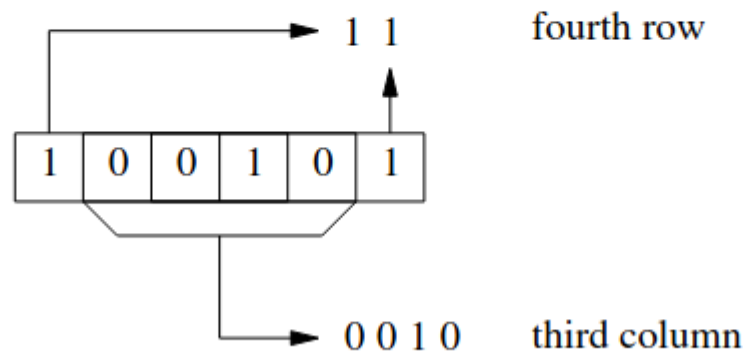
Each S-box contains  $2^6 = 64$  entries, which are typically represented by a table with 16 columns and 4 rows.

Each entry is a 4-bit value.

*S-boxes provide confusion*

# S-boxes

*Example 3.2.* The S-box input  $b = (100101)_2$  indicates the row  $11_2 = 3$  (i.e., fourth row, numbering starts with  $00_2$ ) and the column  $0010_2 = 2$  (i.e., the third column). If the input  $b$  is fed into S-box 1, the output is  $S_1(37 = 100101_2) = 8 = 1000_2$ .



**Fig. 3.12** Example of the decoding of the input  $100101_2$  by S-box 1

| $S_1$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 14 | 04 | 13 | 01 | 02 | 15 | 11 | 08 | 03 | 10 | 06 | 12 | 05 | 09 | 00 | 07 |
| 1     | 00 | 15 | 07 | 04 | 14 | 02 | 13 | 01 | 10 | 06 | 12 | 11 | 09 | 05 | 03 | 08 |
| 2     | 04 | 01 | 14 | 08 | 13 | 06 | 02 | 11 | 15 | 12 | 09 | 07 | 03 | 10 | 05 | 00 |
| 3     | 15 | 12 | 08 | 02 | 04 | 09 | 01 | 07 | 05 | 11 | 03 | 14 | 10 | 00 | 06 | 13 |

| $S_2$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 15 | 01 | 08 | 14 | 06 | 11 | 03 | 04 | 09 | 07 | 02 | 13 | 12 | 00 | 05 | 10 |
| 1     | 03 | 13 | 04 | 07 | 15 | 02 | 08 | 14 | 12 | 00 | 01 | 10 | 06 | 09 | 11 | 05 |
| 2     | 00 | 14 | 07 | 11 | 10 | 04 | 13 | 01 | 05 | 08 | 12 | 06 | 09 | 03 | 02 | 15 |
| 3     | 13 | 08 | 10 | 01 | 03 | 15 | 04 | 02 | 11 | 06 | 07 | 12 | 00 | 05 | 14 | 09 |

| $S_3$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 10 | 00 | 09 | 14 | 06 | 03 | 15 | 05 | 01 | 13 | 12 | 07 | 11 | 04 | 02 | 08 |
| 1     | 13 | 07 | 00 | 09 | 03 | 04 | 06 | 10 | 02 | 08 | 05 | 14 | 12 | 11 | 15 | 01 |
| 2     | 13 | 06 | 04 | 09 | 08 | 15 | 03 | 00 | 11 | 01 | 02 | 12 | 05 | 10 | 14 | 07 |
| 3     | 01 | 10 | 13 | 00 | 06 | 09 | 08 | 07 | 04 | 15 | 14 | 03 | 11 | 05 | 02 | 12 |

| $S_4$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 07 | 13 | 14 | 03 | 00 | 06 | 09 | 10 | 01 | 02 | 08 | 05 | 11 | 12 | 04 | 15 |
| 1     | 13 | 08 | 11 | 05 | 06 | 15 | 00 | 03 | 04 | 07 | 02 | 12 | 01 | 10 | 14 | 09 |
| 2     | 10 | 06 | 09 | 00 | 12 | 11 | 07 | 13 | 15 | 01 | 03 | 14 | 05 | 02 | 08 | 04 |
| 3     | 03 | 15 | 00 | 06 | 10 | 01 | 13 | 08 | 09 | 04 | 05 | 11 | 12 | 07 | 02 | 14 |

# S-boxes

| $S_5$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 02 | 12 | 04 | 01 | 07 | 10 | 11 | 06 | 08 | 05 | 03 | 15 | 13 | 00 | 14 | 09 |
| 1     | 14 | 11 | 02 | 12 | 04 | 07 | 13 | 01 | 05 | 00 | 15 | 10 | 03 | 09 | 08 | 06 |
| 2     | 04 | 02 | 01 | 11 | 10 | 13 | 07 | 08 | 15 | 09 | 12 | 05 | 06 | 03 | 00 | 14 |
| 3     | 11 | 08 | 12 | 07 | 01 | 14 | 02 | 13 | 06 | 15 | 00 | 09 | 10 | 04 | 05 | 03 |

| $S_6$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 12 | 01 | 10 | 15 | 09 | 02 | 06 | 08 | 00 | 13 | 03 | 04 | 14 | 07 | 05 | 11 |
| 1     | 10 | 15 | 04 | 02 | 07 | 12 | 09 | 05 | 06 | 01 | 13 | 14 | 00 | 11 | 03 | 08 |
| 2     | 09 | 14 | 15 | 05 | 02 | 08 | 12 | 03 | 07 | 00 | 04 | 10 | 01 | 13 | 11 | 06 |
| 3     | 04 | 03 | 02 | 12 | 09 | 05 | 15 | 10 | 11 | 14 | 01 | 07 | 06 | 00 | 08 | 13 |

| $S_7$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 04 | 11 | 02 | 14 | 15 | 00 | 08 | 13 | 03 | 12 | 09 | 07 | 05 | 10 | 06 | 01 |
| 1     | 13 | 00 | 11 | 07 | 04 | 09 | 01 | 10 | 14 | 03 | 05 | 12 | 02 | 15 | 08 | 06 |
| 2     | 01 | 04 | 11 | 13 | 12 | 03 | 07 | 14 | 10 | 15 | 06 | 08 | 00 | 05 | 09 | 02 |
| 3     | 06 | 11 | 13 | 08 | 01 | 04 | 10 | 07 | 09 | 05 | 00 | 15 | 14 | 02 | 03 | 12 |

| $S_8$ | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 |
|-------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0     | 13 | 02 | 08 | 04 | 06 | 15 | 11 | 01 | 10 | 09 | 03 | 14 | 05 | 00 | 12 | 07 |
| 1     | 01 | 15 | 13 | 08 | 10 | 03 | 07 | 04 | 12 | 05 | 06 | 11 | 00 | 14 | 09 | 02 |
| 2     | 07 | 11 | 04 | 01 | 09 | 12 | 14 | 02 | 00 | 06 | 10 | 13 | 15 | 03 | 05 | 08 |
| 3     | 02 | 01 | 14 | 07 | 04 | 10 | 08 | 13 | 15 | 12 | 09 | 00 | 03 | 05 | 06 | 11 |

# S-boxes

## Extra reading:

### S-boxes were designed according to the criteria listed below:

1. Each S-box has six input bits and four output bits.
2. No single output bit should be too close to a linear combination of the input bits.
3. If the lowest and the highest bits of the input are fixed and the four middle bits are varied, each of the possible 4-bit output values must occur exactly once.
4. If two inputs to an S-box differ in exactly one bit, their outputs must differ in at least two bits
5. If two inputs to an S-box differ in the two middle bits, their outputs must differ in at least two bits.
6. If two inputs to an S-box differ in their first two bits and are identical in their last two bits, the two outputs must be different.
7. For any nonzero 6-bit difference between inputs, no more than 8 of the 32 pairs of inputs exhibiting that difference may result in the same output difference.
8. A collision (zero output difference) at the 32-bit output of the eight S-boxes is only possible for three adjacent S-boxes.

**Note** that some of these design criteria were not revealed until the 1990s.

S-box introduce a non-linearity in the cipher, i.e.,

$$S(a) \oplus S(b) \neq S(a \oplus b).$$

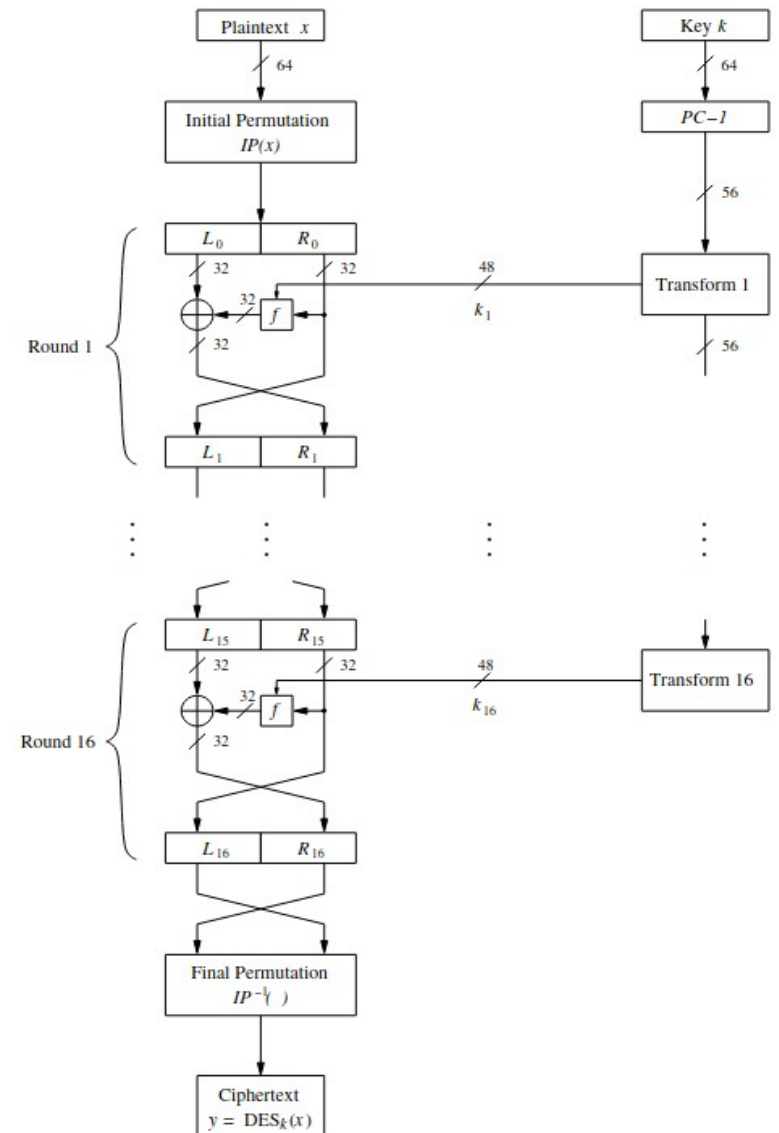
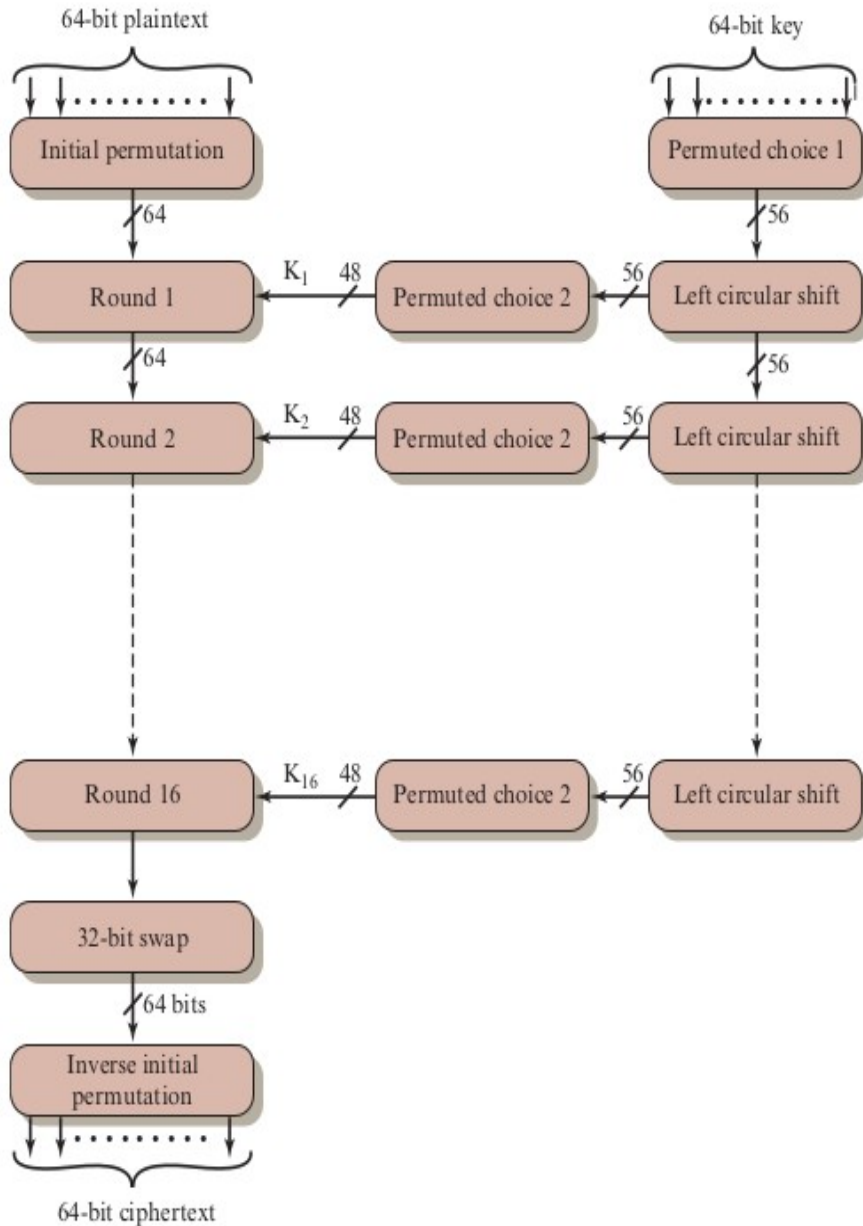
S-boxes were carefully designed to also thwart advanced mathematical attacks, in particular **differential cryptanalysis**

# The permutation $P$ within the $f$ -function

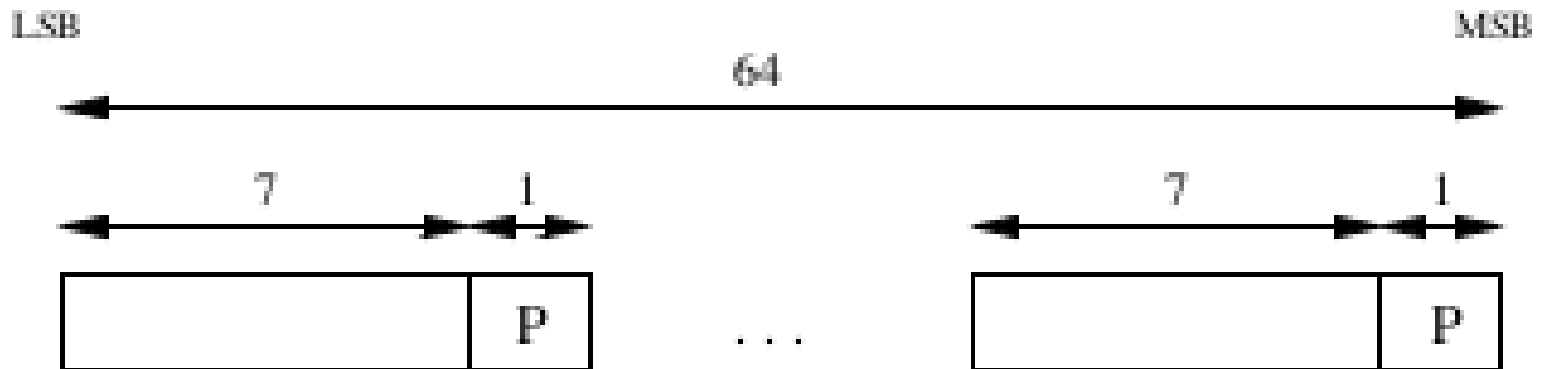
| $P$ |    |    |    |    |    |    |    |
|-----|----|----|----|----|----|----|----|
| 16  | 7  | 20 | 21 | 29 | 12 | 28 | 17 |
| 1   | 15 | 23 | 26 | 5  | 18 | 31 | 10 |
| 2   | 8  | 24 | 14 | 32 | 27 | 3  | 9  |
| 19  | 13 | 30 | 6  | 22 | 11 | 4  | 25 |

- Permutation  $P$  introduces **diffusion** because the **four output bits** of each **S-box** are **permuted** in such a way that **they affect several different S-boxes** in the **following round**.
- The **diffusion** caused by the **expansion**, **S-boxes** and the **permutation  $P$**  guarantees that **every bit at the end of the fifth round** is a function of **every plaintext bit and every key bit**.
- This behavior is known as the **avalanche effect**.

# DES Encryption



# DES Key Schedule



P = parity bit

- Every eighth bit is used as an **odd parity bit** over the preceding seven bits, this does not increase the security.

**Discards bits:** 8, 16, 24, 32, 40, 48, 56, and 64

- DES is a 56-bit key cipher and not a 64-bit.

- This 56-bit is passed to the initial permuted choice (PC-1)

# DES Key Schedule

| $PC - 1$ |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|
| 57       | 49 | 41 | 33 | 25 | 17 | 9  | 1  |
| 58       | 50 | 42 | 34 | 26 | 18 | 10 | 2  |
| 59       | 51 | 43 | 35 | 27 | 19 | 11 | 3  |
| 60       | 52 | 44 | 36 | 63 | 55 | 47 | 39 |
| 31       | 23 | 15 | 7  | 62 | 54 | 46 | 38 |
| 30       | 22 | 14 | 6  | 61 | 53 | 45 | 37 |
| 29       | 21 | 13 | 5  | 28 | 20 | 12 | 4  |

● Initial key permutation, PC-1

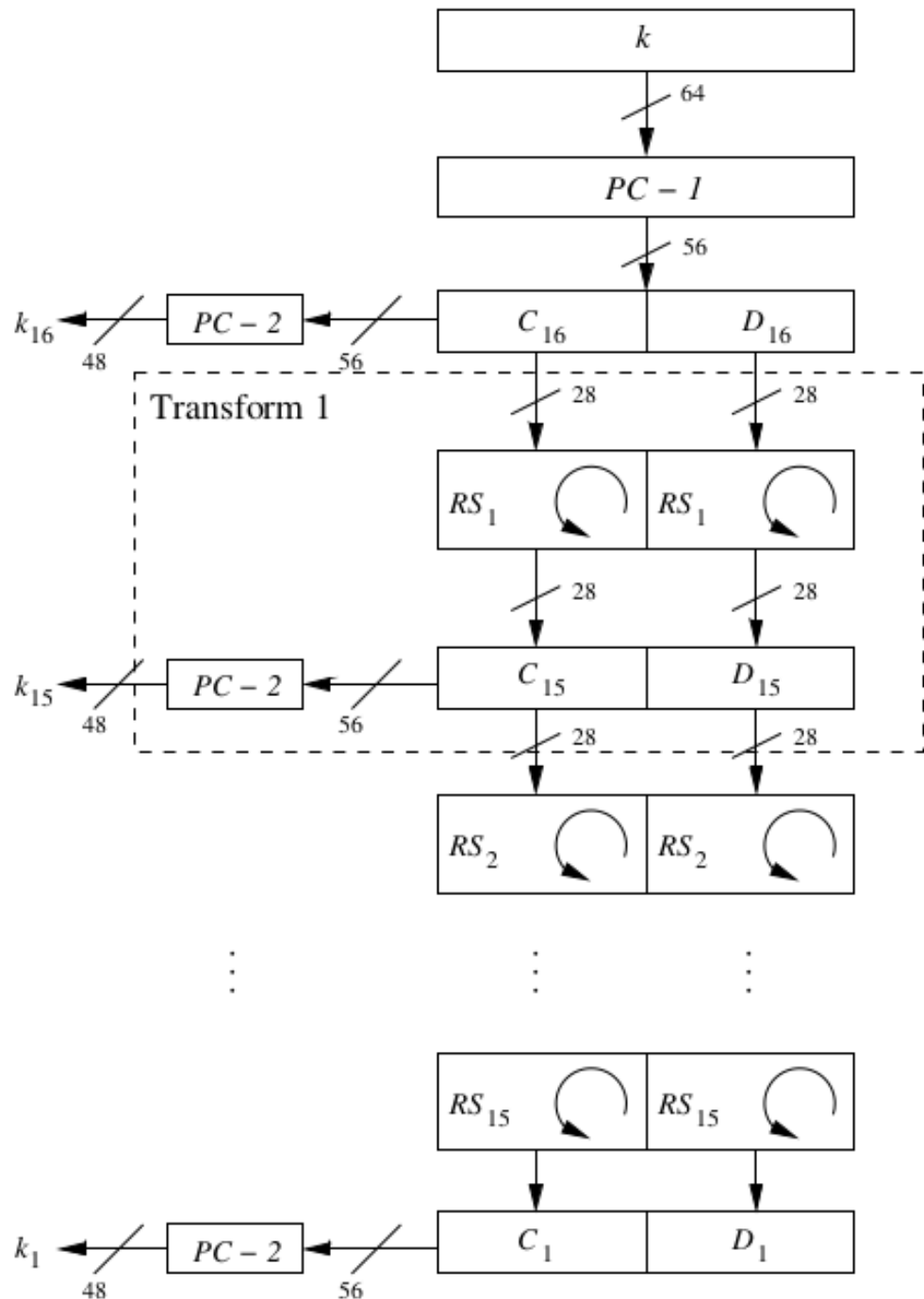
# DES Key Schedule

- The resulting 56-bit key is split into two halves **C0** and **D0**.
- The **two 28-bit halves are** cyclically shifted (**rotated**), left by one or two bit positions depending on the round **i** according to the following rules:
  - In rounds **i = 1, 2, 9, 16**, the two halves are **rotated left by one bit**.
  - In the **other rounds** where  $i \neq 1, 2, 9, 16$ , the two halves **are rotated left by two bits**.

## Note:

- The rotations only take place within either the left or the right half. The total number of rotation positions is  $4 \cdot 1 + 12 \cdot 2 = 28$ .
- This leads to the interesting property that **C0 = C16** and **D0 = D16**.

# DES Key Schedule



# DES Key Schedule

| $PC - 2$ |    |    |    |    |    |    |    |
|----------|----|----|----|----|----|----|----|
| 14       | 17 | 11 | 24 | 1  | 5  | 3  | 28 |
| 15       | 6  | 21 | 10 | 23 | 19 | 12 | 4  |
| 26       | 8  | 16 | 7  | 27 | 20 | 13 | 2  |
| 41       | 52 | 31 | 37 | 47 | 55 | 30 | 40 |
| 51       | 45 | 33 | 48 | 44 | 49 | 39 | 56 |
| 34       | 53 | 46 | 42 | 50 | 36 | 29 | 32 |

- To derive the **48-bit round keys**  $k_i$ , the two halves are permuted bitwise again with PC-2, which stands for “permuted choice 2”.
- **PC – 2 permutes the 56 input bits** coming from  $C_i$  and  $D_i$  and **ignores 8** of them.

# DES Decryption

- Decrypt must unwind steps of data computation
- with Feistel design, do encryption steps again
- using subkeys in reverse order (SK16 ... SK1)
- note that IP undoes final FP step of encryption
- 1st round with SK16 undoes 16th encrypt round
- ....
- 16th round with SK1 undoes 1st encrypt round
- then final FP undoes initial encryption IP
- thus recovering original data value

# Avalanche Effect

- key desirable property of encryption algorithm
- where a change of **one** input or key bit results in changing approx **half** output bits
- making attempts to “home-in” by guessing keys impossible
- DES exhibits strong avalanche

# Strength of DES – Key Size

- 56-bit keys have  $2^{56} = 7.2 \times 10^{16}$  values
- brute force search looks hard
- recent advances have shown is possible
  - in 1997 on Internet in a few months
  - in 1998 on dedicated h/w (EFF) in a few days
  - in 1999 above combined in 22hrs!
- still must be able to recognize plaintext
- now considering alternatives to DES

# Strength of DES – Key Size

**Table 4.5** Average Time Required for Exhaustive Key Search

| Key Size (bits)             | Cipher         | Number of Alternative Keys           | Time Required at $10^9$ Decryptions/s           | Time Required at $10^{13}$ Decryptions/s |
|-----------------------------|----------------|--------------------------------------|-------------------------------------------------|------------------------------------------|
| 56                          | DES            | $2^{56} \approx 7.2 \times 10^{16}$  | $2^{55}$ ns = 1.125 years                       | 1 hour                                   |
| 128                         | AES            | $2^{128} \approx 3.4 \times 10^{38}$ | $2^{127}$ ns = $5.3 \times 10^{21}$ years       | $5.3 \times 10^{17}$ years               |
| 168                         | Triple DES     | $2^{168} \approx 3.7 \times 10^{50}$ | $2^{167}$ ns = $5.8 \times 10^{33}$ years       | $5.8 \times 10^{29}$ years               |
| 192                         | AES            | $2^{192} \approx 6.3 \times 10^{57}$ | $2^{191}$ ns = $9.8 \times 10^{40}$ years       | $9.8 \times 10^{36}$ years               |
| 256                         | AES            | $2^{256} \approx 1.2 \times 10^{77}$ | $2^{255}$ ns = $1.8 \times 10^{60}$ years       | $1.8 \times 10^{56}$ years               |
| 26 characters (permutation) | Monoalphabetic | $26! = 4 \times 10^{26}$             | $2 \times 10^{26}$ ns = $6.3 \times 10^9$ years | $6.3 \times 10^6$ years                  |

***Cryptography and Network Security by William Stallings, 8<sup>th</sup> Edition***

# Strength of DES – Timing Attacks

Also called side-channel attacks

attacks actual implementation of cipher

- use knowledge of consequences of implementation to derive knowledge of some/all subkey bits
- specifically use fact that calculations can take varying times depending on the value of the inputs to it
- particularly problematic on smartcards

# Strength of DES – Analytic Attacks

- now have several analytic attacks on DES
- these utilise some deep structure of the cipher
  - by gathering information about encryptions
  - can eventually recover some/all of the sub-key bits
  - if necessary then exhaustively search for the rest
- generally these are statistical attacks
- include
  - differential cryptanalysis
  - linear cryptanalysis
  - related key attacks

# Block Cipher Design Principles

- basic principles still like Feistel in 1970's
- number of rounds
  - more is better, exhaustive search best attack
- function  $f$ :
  - provides “confusion”, is nonlinear, avalanche
- key schedule
  - complex subkey creation, key avalanche

# Summary

- have considered:
- block cipher design principles
- DES
  - Details
  - Strength
  - Attacks

# Reference

- **Text book:** Understanding Cryptography: A Textbook for Students and Practitioners by Christof Paar, and Jan Pelzl
- Download from here while it is still freely available:

<https://gnanavelrec.wordpress.com/wp-content/uploads/2019/06/2.understanding-cryptography-by-christof-paar-.pdf>